

Neural Networks, Explained Like You're Training a New Hire

The one-sentence version: A neural network is a system that gets better at making decisions the same way a new employee does — by seeing lots of examples and getting corrected when it's wrong.

That's it. Everything else below is just unpacking that sentence.

Meet your new hire

Picture a new employee reviewing requests and deciding: *approve* or *flag for review*. On day one they just take in whatever they're given — the request, some background, a few context clues — and guess. That's the **input layer**.

Then comes the weighing-it-out moment: *"This detail matters more than that one... this feels off."* That's the **hidden layer** — a bunch of small, partial judgments happening at once, each paying more or less attention to different pieces of the input.

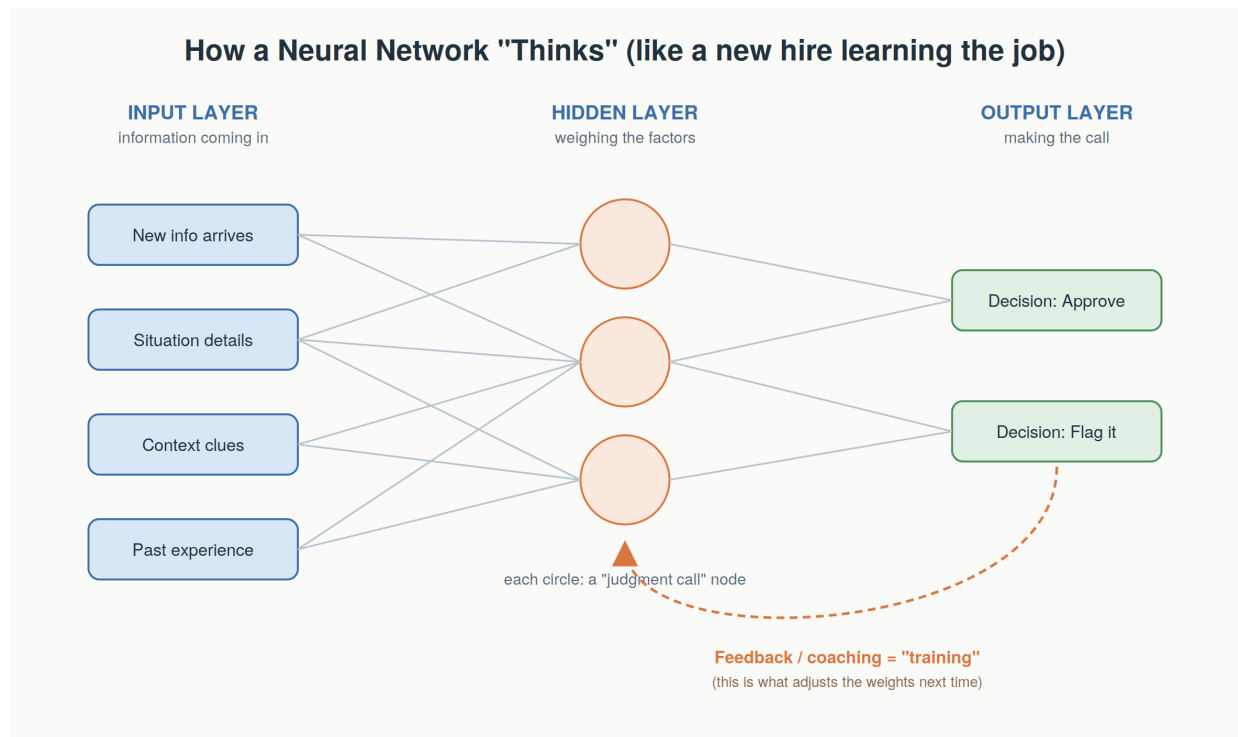
Finally, they land on a decision. That's the **output layer**.

How the “employee” actually learns

This is what makes it a *neural network* and not just a flowchart: every time the decision is wrong, a manager corrects them, and the employee quietly adjusts how much weight they give to each factor. Not a lecture — repetition and feedback.

That correction process is **training**. Each adjustment is a **weight**. The technical term for “adjust the weights based on how wrong the answer was” is **backpropagation** — but really it's just: *get it wrong, get told why, adjust slightly, repeat thousands of times*.

Laid out visually:



Neural network diagram: an input layer feeding a hidden layer of judgment nodes, feeding an output decision, with a feedback loop labeled training.

Why this matters

The catch — for the employee and for a real neural network — is that after enough training, *they* can't fully explain *why* they made a call. The judgment lives in thousands of small, adjusted weights, not one clean rule. That's why neural networks get called "black boxes": explaining *how* they decided is harder than explaining *what* they decided.

Tools used

I used Claude (Anthropic) to help build an analogy that holds up end-to-end (input → processing → output → learning) and to generate the diagram, then reviewed and edited it myself so the analogy stayed accurate and didn't lose the "black box" limitation in the name of simplicity.